

Error Log

Noorinder

Credit Name: Chapter 13

Assignment Name: ReverseList

1. Missing Stack Initialization

- **Error:** The stack was not initialized before being used, causing a `NullPointerException` when attempting to push elements.
- **Cause:** The `Stack` object wasn't created before performing stack operations like `push()`.
- **Fix:** Initialized the stack before collecting user inputs.

Before Fix:

```
stack.push(num); // Causes NullPointerException as stack is not initialized
```

After Fix:

```
// Initialize the stack before use
Stack<Integer> stack = new Stack<>();
stack.push(num); // Now works correctly
```

2. Lack of Input Validation

- **Error:** The program did not validate user inputs, allowing non-integer values to cause runtime exceptions.
- **Cause:** The `Scanner` was used without checking if the input was an integer, leading to invalid inputs being processed.
- **Fix:** Added a check to validate user input, prompting the user to try again if the input was invalid.

Before Fix:

```
int num = input.nextInt(); // Assumes all inputs are valid integers
stack.push(num);
```

After Fix:

```
if (input.hasNextInt()) { // Check if the input is an integer
    int num = input.nextInt();
    stack.push(num); // Push valid numbers onto the stack
} else {
    System.out.println("Invalid input. Please enter an integer.");
    input.next(); // Clear the invalid input
}
```

3. Exiting Without Proper Notification

- **Error:** The program stopped collecting inputs without informing the user why (e.g., maximum size reached or quit command entered).
- **Cause:** There were no messages to explain why input collection stopped.
- **Fix:** Added user notifications to indicate whether the stack was full or input was manually stopped.

Before Fix:

```
if (stack.size() == 10 || num == 999) {
    break; // No notification to user
}
```

After Fix:

```
if (stack.size() == 10) {
    System.out.println("Maximum of 10 numbers reached. Stopping input.");
    break;
} else if (num == 999) {
    System.out.println("Input stopped by user.");
    break;
}
```

4. Inefficient Reverse Output

- **Error:** The program printed numbers in reverse directly without storing them, limiting flexibility for further operations or formatting.
- **Cause:** The reversed output was processed on the fly and not retained for additional functionality.

- **Fix:** Collected reversed numbers into a list before displaying them.

Before Fix:

```
while (!stack.isEmpty()) {  
    System.out.println(stack.pop()); // Prints numbers directly  
}
```

After Fix:

```
List<Integer> reversedNumbers = new ArrayList<>();  
while (!stack.isEmpty()) {  
    reversedNumbers.add(stack.pop()); // Store numbers in a list  
}  
System.out.println("Numbers in reverse: " + reversedNumbers);
```

5. Resource Management

- **Error:** The `Scanner` object wasn't closed, leading to potential resource leaks.
- **Cause:** The `Scanner` was left open after program termination.
- **Fix:** Explicitly closed the `Scanner` at the end of the program.

Before Fix:

```
// Scanner not closed
```

After Fix:

```
input.close(); // Close the Scanner to release resources
```